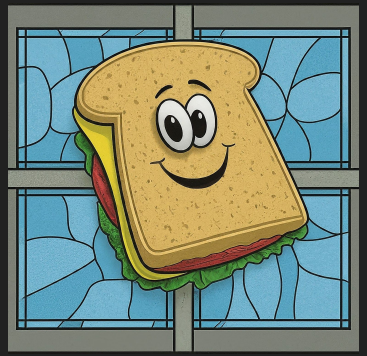
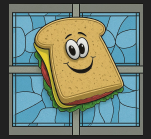


Sudos and Sudon’ts

Internals, plus a couple vulns

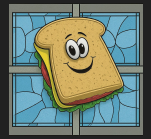
Michael “mtu” Torres <tmux@google.com>

Overview



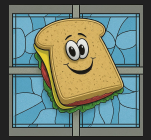
- Introduction
- Initial Research
- First Issue - Memory Safety
- Second Issue - Search path weirdness
- ALPC Analysis
- First Vuln - Insufficient Client Auth
- Second Vuln - Tampering
- Final Thoughts

Introduction - Me



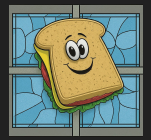
- Operational technology security at Google
- Cyber (sort of) for the U.S. Marine Corps Reserve
- Volunteer with VetSec (veteransec.org)
- I do random research projects like this when I get bored

Introduction - Sudo for Windows



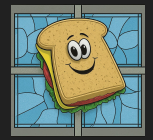
- Expected in Windows 11 24H2 update
  - Currently in Insider Preview
- Coordinates process elevation (via UAC) to support CLI features
    - Console Input/Output
    - STDIN/STDOUT/STDERR
    - Pipes and redirection
- Written in Rust, open source
- Two issues with minimal/no security impact
  - Memory corruption in path resolution
  - Search order confusion
- Two issues with security impact
  - Cross-user code execution
  - Data tampering (details still in embargo)

Introduction - Sudo for Windows



- Most research conducted against Sudo v1.0.0
  - Signature dated 13 February 2024
- Still-embargoed issue confirmed against a **different** Sudo v1.0.0
  - Signature dated 4 March 2024

Initial Research - Precursors

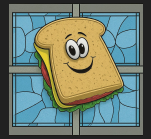


- User Account Control (UAC) [1]
  - “Normal” processes run in medium integrity
    - ~everything not explicitly made to run at low integrity (like most web browsers)
  - “Elevated” processes run in high integrity
    - Required to modify system data (aka get SYSTEM permissions)
  - Considered a “defense in depth” measure, not a primary control [2]
    - Not serviced by MSRC
  - Cannot inherit handles from parent processes
    - Console Input/Output, STDIN/OUT/ERR

[1] https://learn.microsoft.com/en-us/windows/security/application-security/application-control/user-account-control/how-it-works

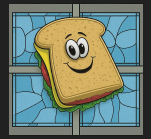
[2] https://www.microsoft.com/en-us/msrc/windows-security-servicing-criteria

Initial Research - Precursors

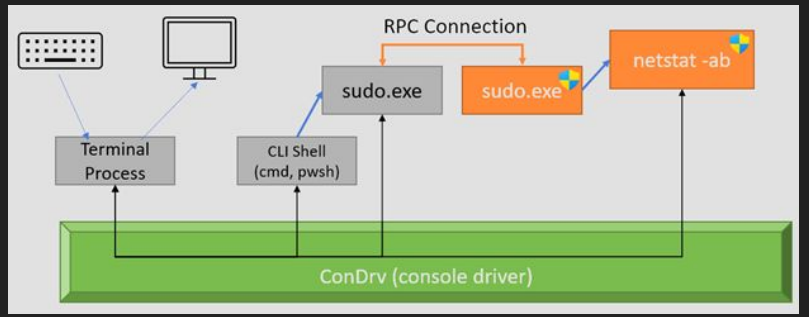


- Advanced Local Procedure Call (ALPC)
  - Interprocess communications (IPC)
  - Undocumented (officially)
  - Server listens on a “Port” object in the `RPC Control` object tree
    - Ports are string values (e.g. `sudo_elevate_1234` or a string GUID)
  - Server can query for attributes of the client (PID, principal name)
  - Client can set requirements for server identity (SID or other principal name)
- Windows APIs in Rust
  - `windows` and `windows-sys`
  - Both require `unsafe` to call APIs
  - `windows` will do type coercion for you

Initial Research

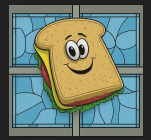


- Blog post [1] has some good details
  - Three modes
    - New Window - most secure, basically just a `runas` wrapper
    - Input Closed - Elevated process can't see unelevated console's input
    - Inline - Elevated process gets access to unelevated console input/output
  - Open source [2], but it wasn't when I did most of this research)':
  - Diagram to explain request flow



[1] https://devblogs.microsoft.com/commandline/introducing-sudo-for-windows/
[2] https://github.com/microsoft/sudo/tree/main

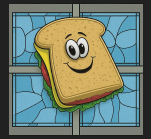
Initial Research



- “New Window” doesn’t always avoid creating the RPC server [1]
  - If requester has manual CWD or environment preservation, RPC server is still needed
  - The “most secure” execution method is still vulnerable to any issues involving the RPC server
  - It also gets STDIN handle, unlike “Input Disabled”

[1] https://github.com/microsoft/sudo/blob/main/sudo/src/run_handler.rs#L313-L333

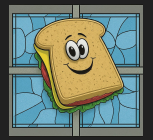
Initial Research



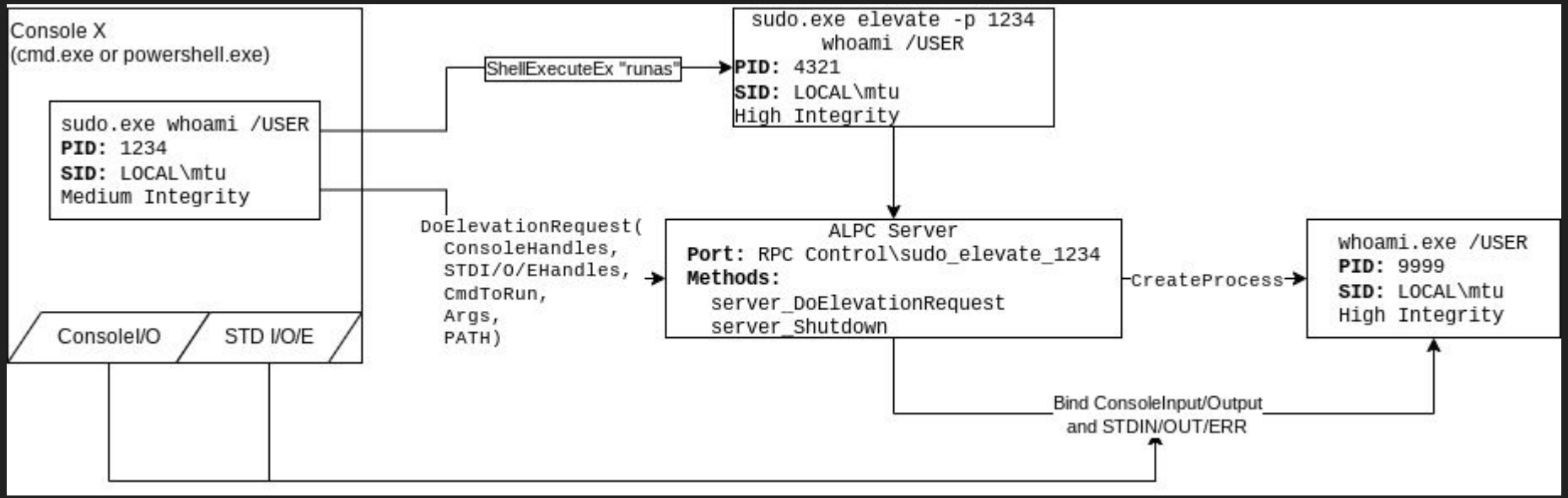
- James Forshaw's 9 Feb 2024 blog post [1]
  - Which is also what triggered my interest in this target
  - At the time, the privileged RPC server had no authentication
    - "Fixed" by the time I was testing
  - Highlighted the RPC methods of interest
    - `server_DoElevationRequest`
    - Passes handles to STDIN/OUT/ERR and Console Input/Output
    - Passes the parameters for the command to be ran
      - Application
      - Arguments
      - CWD
      - Environment

[1] https://www.tiraniddo.dev/2024/02/sudo-on-windows-quick-rundown.html

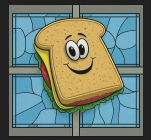
Initial Research



- More detailed look at runmode 3, Inline



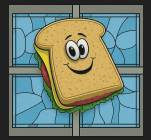
Initial Research



Summary

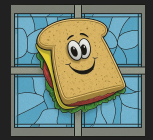
- Written in Rust and open-sourced
  - Rust requires `unsafe` to call Windows APIs
- ALPC is used to coordinate elevation using a unelevated client and an elevated server, both running as the user
  - Used to be unauthenticated
  - Client passes all the parameters used to run the elevated proccess

First Issue - Memory safety



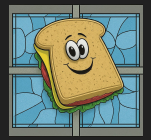
- In a Rust program!
- Path resolution for non-standard leading characters
- I don’t have source code (and git history wasn’t preserved)
- Best guess on root cause
  - Assumption that `path.to_str()` handled zero-termination at path length
    - It’s really just a vector of bytes, with the `length` parameter set to where the null byte should be
  - Passing `path.to_str().as_ptr()` resulted in reading until the first null, not until the end of the parsed path

First Issue - Memory safety



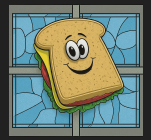
```rust
use std::{env, path::PathBuf};
use  windows_sys::Win32::Storage::FileSystem::GetBinaryTypeA;


fn main() {
    let args: Vec<String> = env::args().collect();

    let mut path = PathBuf::from(r"C:\Users\mtu\");
    println!("Base path: {path:?}");

    println!("Pushing path: \"{}\"", args[1]);
    path.push(args[1].as_str());

    println!("Path after push: {:?}", path);

    let mut output_binary_type: u32 = 0;
    let path_as_str = path.to_str().unwrap();
    let path_ptr = path_as_str.as_ptr() as *const u8;

    unsafe {
        let reconverted_path_str = std::ffi::CStr::from_ptr(path_ptr as *const i8);
        println!("GetBinaryTypeA({:#?}, {:08x})", reconverted_path_str.to_str().unwrap(), output_binary_type);
        GetBinaryTypeA(path_ptr, &mut output_binary_type as *mut u32);
    }
}
```

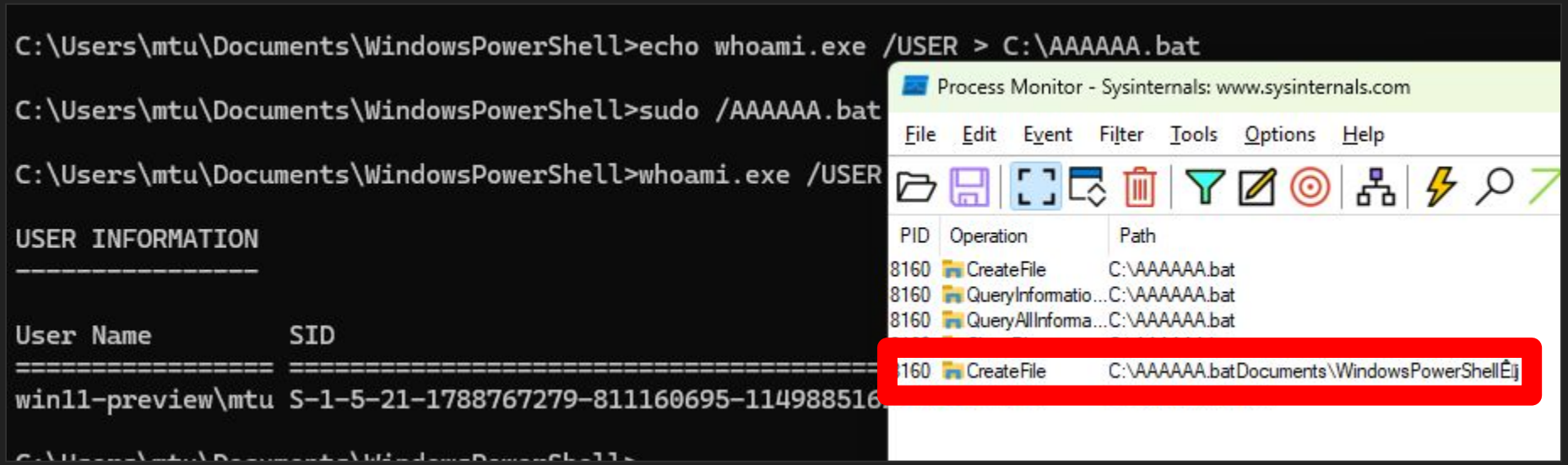
First Issue - Memory safety



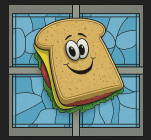
- Can be used to “read” non-null bytes from old heap allocations
- Can’t be used to overwrite arbitrary memory
  - Backing data type is a `Vec[u8]`, so it automatically will realloc for growth
- Untrusted `sudo.exe` arguments could result in running an unexpected program
  - …but you shouldn’t be running sudo with untrusted arguments.

First Issue - Memory safety



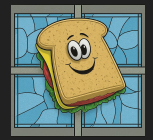


Second Issue - Search Order Inconsistency



- A user would expect that `sudo program.exe` would run the executable in the active shell's current working directory (`$CWD\program.exe`)
- Path resolution occurred within the privileged process, using the unprivileged PATH last
- In a somewhat contrived example, if you tried to run a trusted, self-compiled process `cmd.exe` in the current directory, it would spawn the CMD shell
- According to Microsoft - not a vulnerability
    - "In scenarios where a user has administrator privileges or an administrator has authenticated in a standard user session, the system is already compromised, negating the reported vulnerability."

Second Issue - Search Order Inconsistency

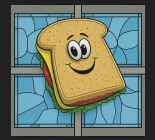


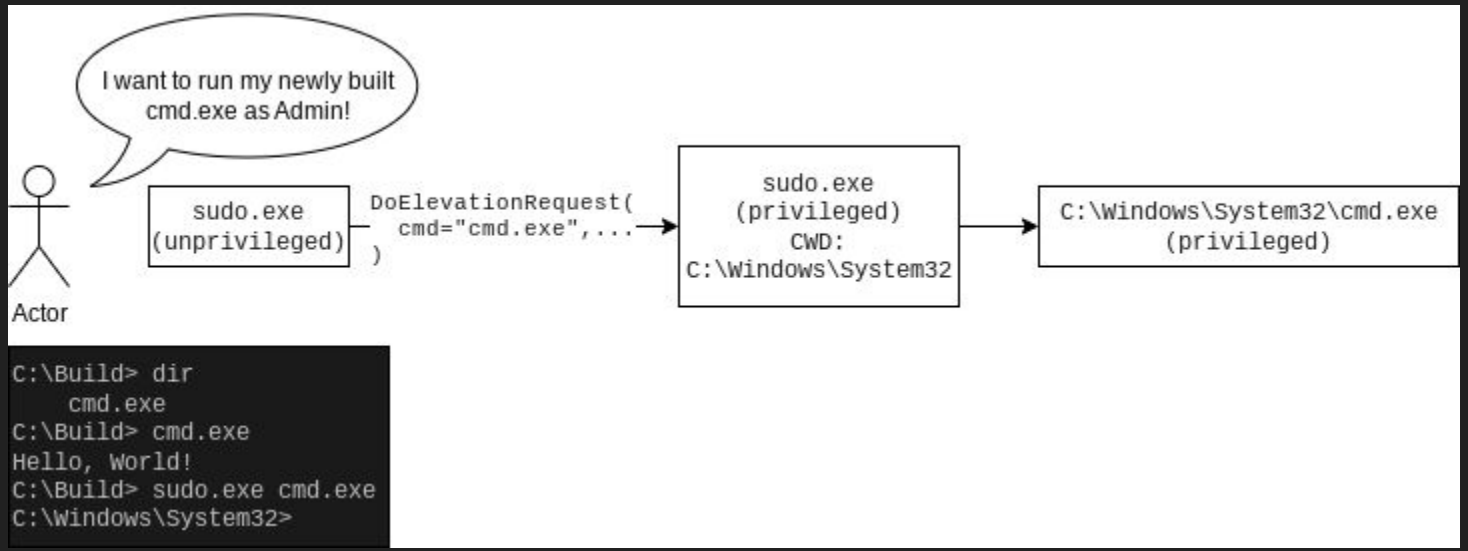
- But they fixed it anyway 🥲 [1]
- Check if `which` can resolve the program name, and replaced the requested program name with the full path if it can.

[1] https://github.com/microsoft/sudo/blob/main/sudo/src/run_handler.rs#L264-276

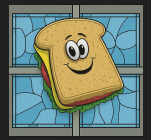
```rust
// Does the application exist somewhere on the path?
let where_result = which::which(&req.application);

if let Ok(path) = where_result {
    // It's a real file that exists on the PATH.

    // Replace the request's application with the full path to the exe. This
    // ensures that the elevated sudo will execute the same thing that was
    // found here in the unelevated context.

    req.application = absolute_path(&path)?.to_string_lossy().to_string();
    adjust_args_for_gui_exes(&mut req);
} else {
```

Second Issue - Search Order Inconsistency





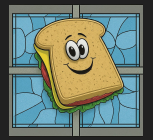
RPC Server Analysis

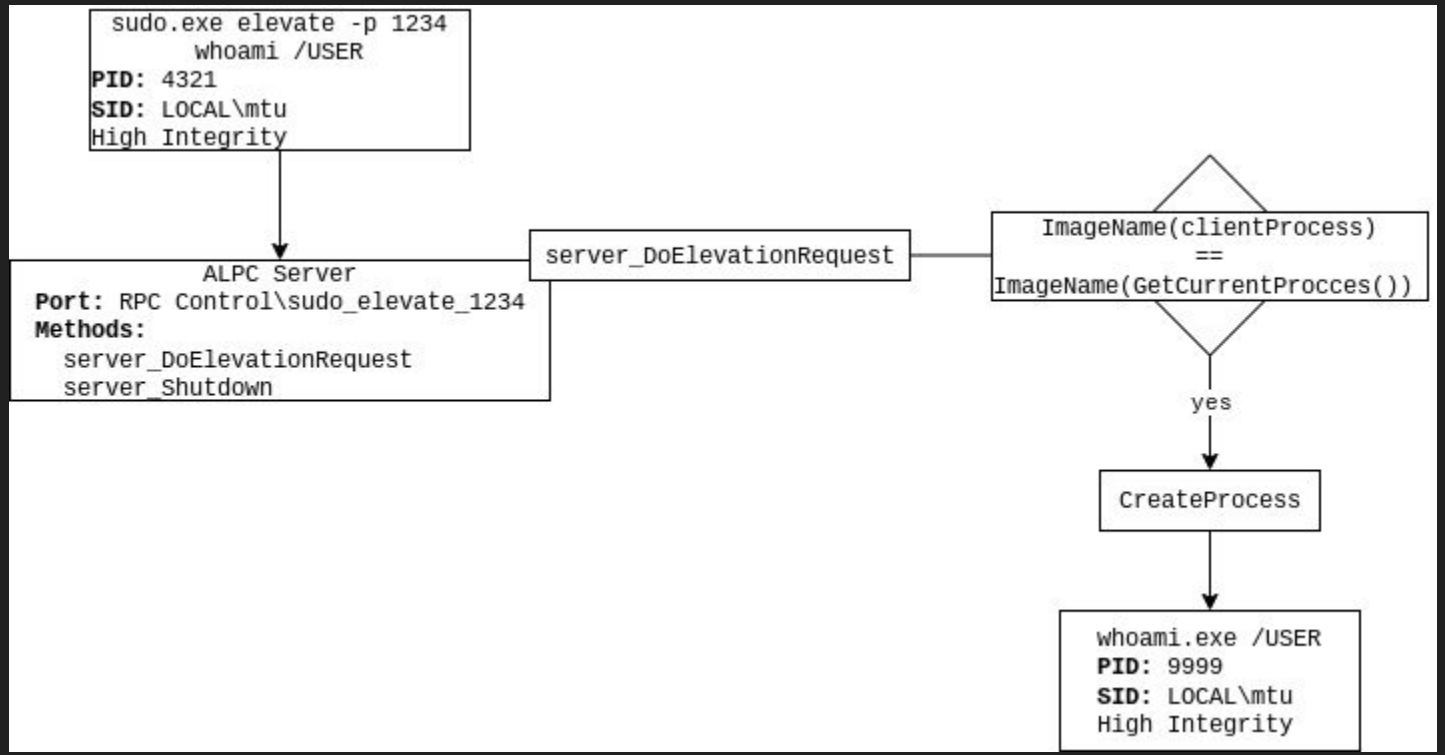


- Asynchronous Local Procedure Call (ALPC)
- Static analysis to determine parameters for the elevation request
- Unprivileged PID is used as part of the ALPC socket
  - `RPC Control\sudo_elevate_$clientPID`
- After James' blog post, an auth check was added
  - Check that the client's full process path is the same as the server's full process path
  - Effectively, client *must* be an instance of `sudo.exe`

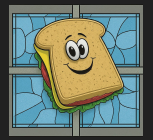
```
int server_DoElevationRequest(
    HANDLE hRpc,
    HANDLE hInputProcess,
    HANDLE hPipe,
    HANDLE hFile,
    int iRunMode,
    rust_str *cmd,
    rust_str *args,
    rust_str *cwd,
    rust_str *environment,
    GUID *guid,
    HANDLE hOutputProcess);
```

RPC Server Analysis



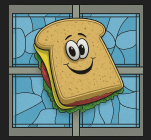


Vulnerability 1 - Insufficient Client Authentication



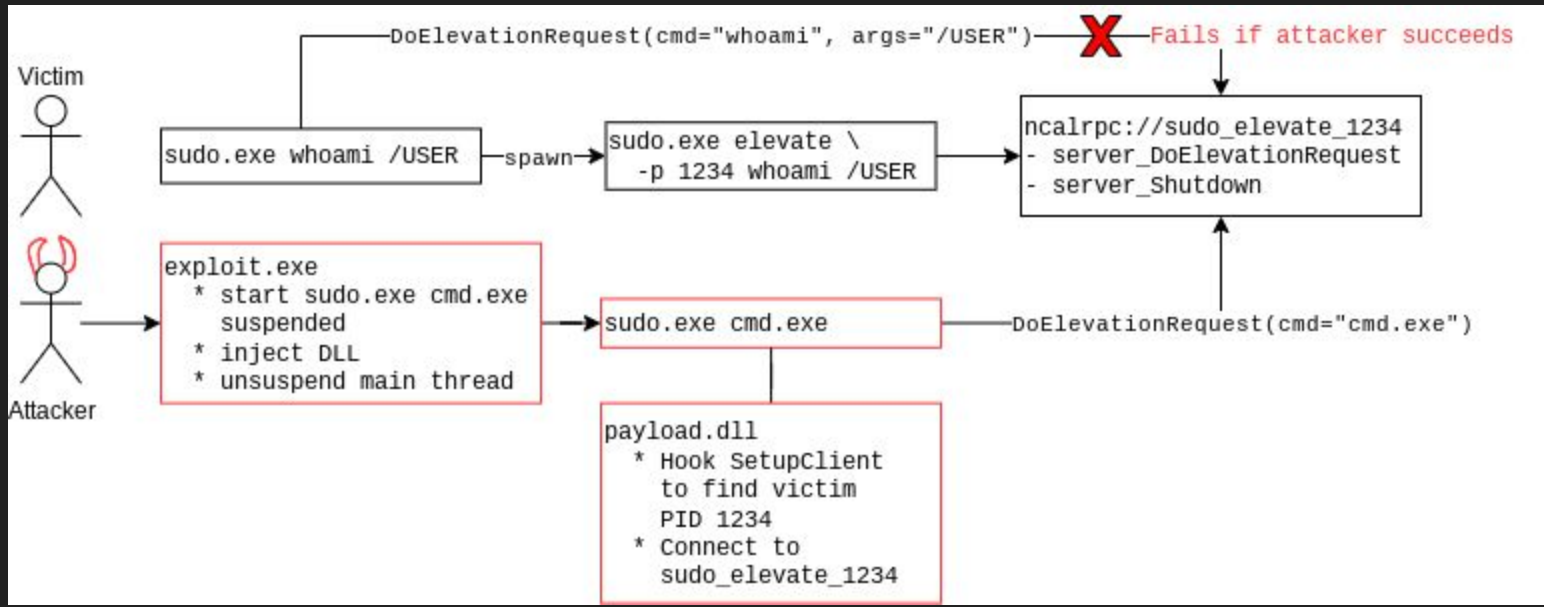
- RPC server (privileged process) does not properly authenticate that the client is the legitimate client
- Any user (incl. sandboxed processes) can issue elevation requests to Sudo RPC servers
- **Result:** Using sudo can result in a local attacker running arbitrary commands as the sudo user.

Vulnerability 1 - Insufficient Client Authentication

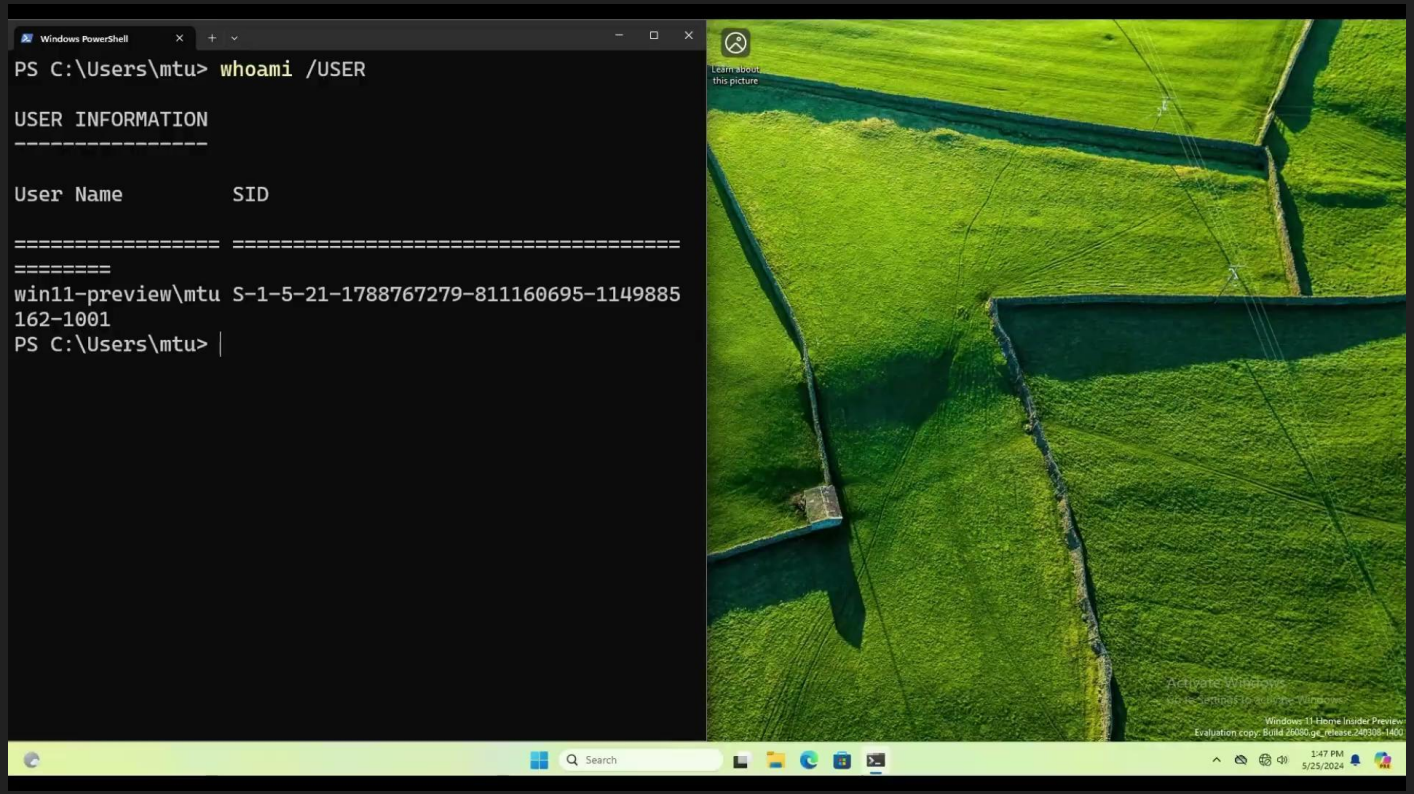


- Exploitation needs two things
  - Insufficient authentication - any user can run arbitrary code as any image name
    - PoC used DLL injection/function hooking to manipulate a `sudo.exe` process
  - Predictable RPC port name
    - Users can view the PIDs of other users' processes
- Attacker-controlled code in a `sudo.exe` process to call `server_DoElevationRequest` with arbitrary arguments
  - Legitimate RPC server will happily run the attacker-controlled executable as a privileged user
- Patched in Sudo version released 4 March 2024

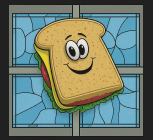
Vulnerability 1 - Insufficient Client Authentication



Vulnerability 1 - Insufficient Client Authentication



Vulnerability 1 - Insufficient Client Authentication

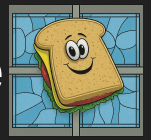


- Found out later - they make the race window reaaaaally short):

```
// Only the first caller will get their request handled. Everyone else will
// be forced to bail out.
if RPC_SERVER_IN_USE.swap(true, Ordering::Relaxed) {
    // We're already in the middle of handling a request.
    return ERROR_BUSY.to_hresult();
}
```

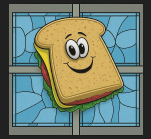
https://github.com/microsoft/sudo/blob/main/sudo/src/rpc_bindings_server.rs#L195-L200

Vulnerability 2 - Data tampering/Information Disclosure

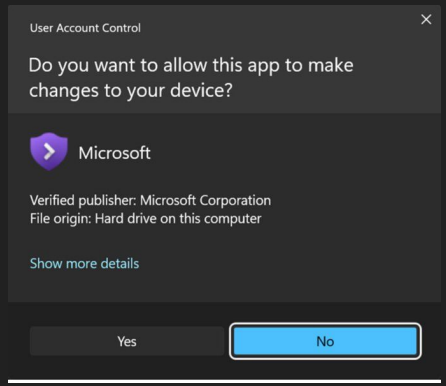


- Reported 13 June 2024
- Still in embargo period pending a fix or 90 day delay

Public Issues of Note

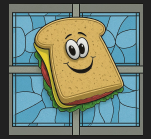


- Elevation prompt is confusing [1]
  - It's hard to tell what hitting "yes" will do
  - "Verified Publisher" status is misleading, because sudo can end up running unverified code
- Sudo won't insult you for failing to elevate [2]



[1] https://github.com/microsoft/sudo/issues/16
[2] https://github.com/microsoft/sudo/issues/31

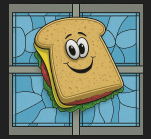
Summary



- Sudo for Windows is coming soon™
  - Open source!
  - Lets users elevate CLI programs in a way that keeps inputs and outputs available
  - Uses UAC and inter-process RPCs to manage process elevation
- Unsafe Rust is required to call Windows APIs
  - Meaning that any program that calls APIs can get it wrong

- Questions?

References



- Introducing Sudo for Windows - Microsoft Dev Blog
  - https://devblogs.microsoft.com/commandline/introducing-sudo-for-windows/
- Sudo for Windows - Microsoft Learn
  - https://learn.microsoft.com/en-us/windows/sudo/
- Sudo - GitHub Project
  - https://github.com/microsoft/sudo
- Sudo on Windows - Quick Rundown - James Forshaw
  - https://www.tiraniddo.dev/2024/02/sudo-on-windows-quick-rundown.html
- NtObjectManager - PowerShell Gallery
  - https://www.powershellgallery.com/packages/NtObjectManager/2.0.1
- Rust for Windows - GitHub Project
  - https://github.com/microsoft/windows-rs